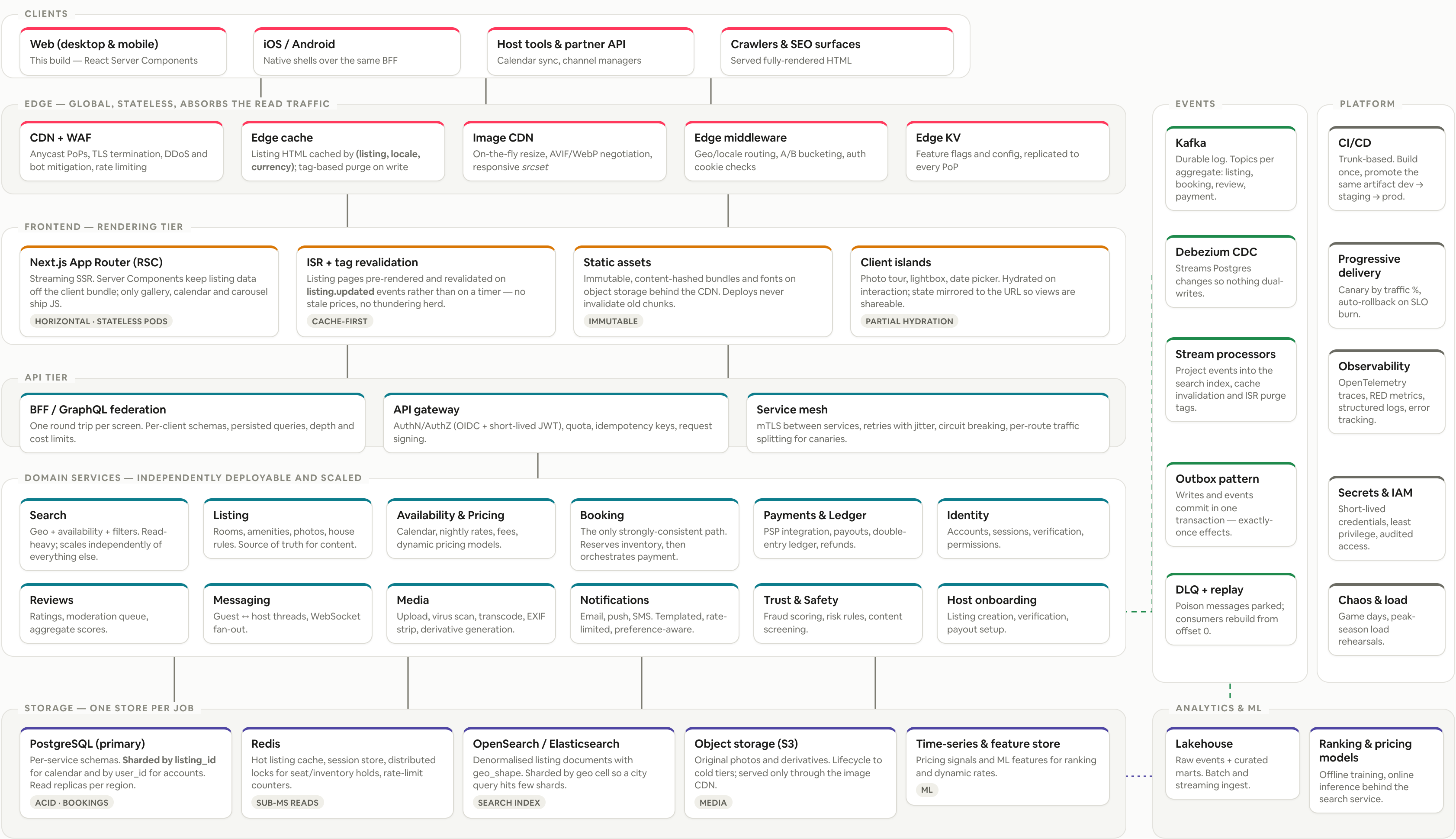


Vacation-Rental Marketplace — Production Architecture

Scaling strategy: frontend · backend · storage · search · deployment



Frontend scaling

Reads dominate ~1000:1. Listing pages are **statically rendered and edge-cached**, so traffic is absorbed at the PoP and origin pods scale with writes, not views. RSC keeps the client bundle small; only three interactive islands ship JS.

Search scaling

Search never queries Postgres. A denormalised document per listing is projected from the event log and **sharded by geo cell**, so a city-level query touches a handful of shards. Popular queries are cached with a short TTL.

Booking correctness

The one path that cannot be eventually consistent. Inventory is held with a **row-level lock plus an idempotency key**, then payment runs as a **saga** with compensating refunds. Everything downstream reacts to events.

Storage tiering

Hot metadata in Redis, transactional truth in sharded Postgres, media in object storage behind an image CDN. **CDC — never dual writes** — keeps the search index, caches and warehouse in step.

Deployment

Multi-region active-active for reads, with a single writer region per booking shard to keep inventory linearizable. Canary releases gated on SLO burn rate; every artifact is immutable and rollback is a pointer move.